

Memoria técnica

Motor de búsqueda de libros Stage 1: la capa de datos

Implementación en Java: arquitectura, clases, ejecución y benchmarks

Autor	Daniel Rodríguez Alonso (implementación Java)
Asignatura	Big Data · 4º curso · ULPGC
Repositorio	github.com/BigData-Ulpgc/stage_1 · rama <i>java</i>
Estado	Retos 1-31 completados · 354 tests en verde
Fecha	2 de octubre de 2026

Este documento está pensado para estudiar: explica qué hace cada pieza, por qué está diseñada así, cómo se ejecuta todo y qué dicen los benchmarks. Al final hay una batería de preguntas de defensa con sus respuestas.

Índice

1. Qué es el proyecto	4
1.1 Qué hace el sistema, en una frase por capa	4
1.2 El contrato común (shared/SPEC.md)	4
2. Arquitectura	5
2.1 Recorrido completo de un libro	5
2.2 Capas y dependencias	6
2.3 Principios de diseño que se repiten	6
3. Organización del proyecto	7
3.1 Árbol de carpetas	7
3.2 Cifras	7
3.3 Dónde vive cada parte del estado	7
4. Cómo se ejecuta	8
4.1 Requisitos	8
4.2 Compilar y pasar los tests	8
4.3 La aplicación: comandos de Main	8
4.4 Configuración: cambiar de estructura sin tocar código	9
4.5 Los benchmarks	9
5. Clases y responsabilidades	10
5.1 Paquete model: los datos que viajan	10
5.2 Paquete crawler: descargar y separar	10
5.3 Paquete datalake: guardar los libros	11
5.4 Paquete datamart.metadata: metadatos en SQLite	12
5.5 Paquete datamart.index: el índice invertido	13
5.6 Paquete query: la búsqueda	15
5.7 Paquete control: progreso y recuperación	15
5.8 Paquete config y ensamblado	16
5.9 Paquete benchmark	17
6. Estrategia de pruebas	18
6.1 La prueba end-to-end (reto 31)	18
7. Benchmarks	19
7.1 Metodología común	19
7.2 Datos: libros reales y sintéticos	19
7.3 Benchmark del datalake (reto 27)	20
7.4 Benchmark de metadatos (reto 28)	21
7.5 Benchmark de los índices invertidos (reto 29)	23
7.6 Datos reales frente a sintéticos: qué cambia	26

7.7 Resumen: qué estructura elegir	27
8. Preguntas de defensa	28
8.1 Diseño general	28
8.2 Robustez y recuperación	28
8.3 Índice y búsqueda	29
8.4 Benchmarks	29
8.5 Pruebas	30
9. Limitaciones y trabajo futuro	31

1. Qué es el proyecto

El objetivo del Stage 1 es construir la **capa de datos** de un motor de búsqueda de libros del Project Gutenberg. No se busca todavía «bonito» (sin ranking, sin interfaz web): se busca que los datos se descarguen, se guarden, se describan y se indexen de forma **correcta, recuperable tras un fallo y medible**, y comparar varias formas de organizarlos.

El grupo implementa lo mismo en tres lenguajes (Java, Python y C). Esta memoria cubre la implementación en **Java**. Para que las tres sean comparables existe un contrato común, `shared/SPEC.md`, que fija las reglas que no pueden variar entre lenguajes.

1.1 Qué hace el sistema, en una frase por capa

Capa	Qué hace	Dónde (paquete)
Ingesta	Descarga el texto de Gutenberg y lo separa en <i>header</i> (cabecera con título, autor...) y <i>body</i> (el libro en sí); el pie de licencia se descarta.	<code>crawler</code>
Datalake	Guarda header y body tal cual, en ficheros. Tres organizaciones posibles: book , range y time .	<code>datalake</code>
Datamarts	Datos ya procesados para consultar: metadatos en SQLite y el índice invertido (término → libros) en tres variantes.	<code>datamart.*</code>
Consulta	Búsqueda AND: libros que contienen <i>todos</i> los términos de la consulta.	<code>query</code>
Control	Ficheros que recuerdan qué libros ya se descargaron e indexaron, para no repetir trabajo y poder continuar tras un corte.	<code>control</code>
Configuración	Un único sitio para rutas, timeouts y estructuras activas; factories que traducen nombres a implementaciones.	<code>config</code>
Benchmarks	Miden y comparan las estructuras con una metodología común y escriben CSV.	<code>benchmark</code>

1.2 El contrato común (shared/SPEC.md)

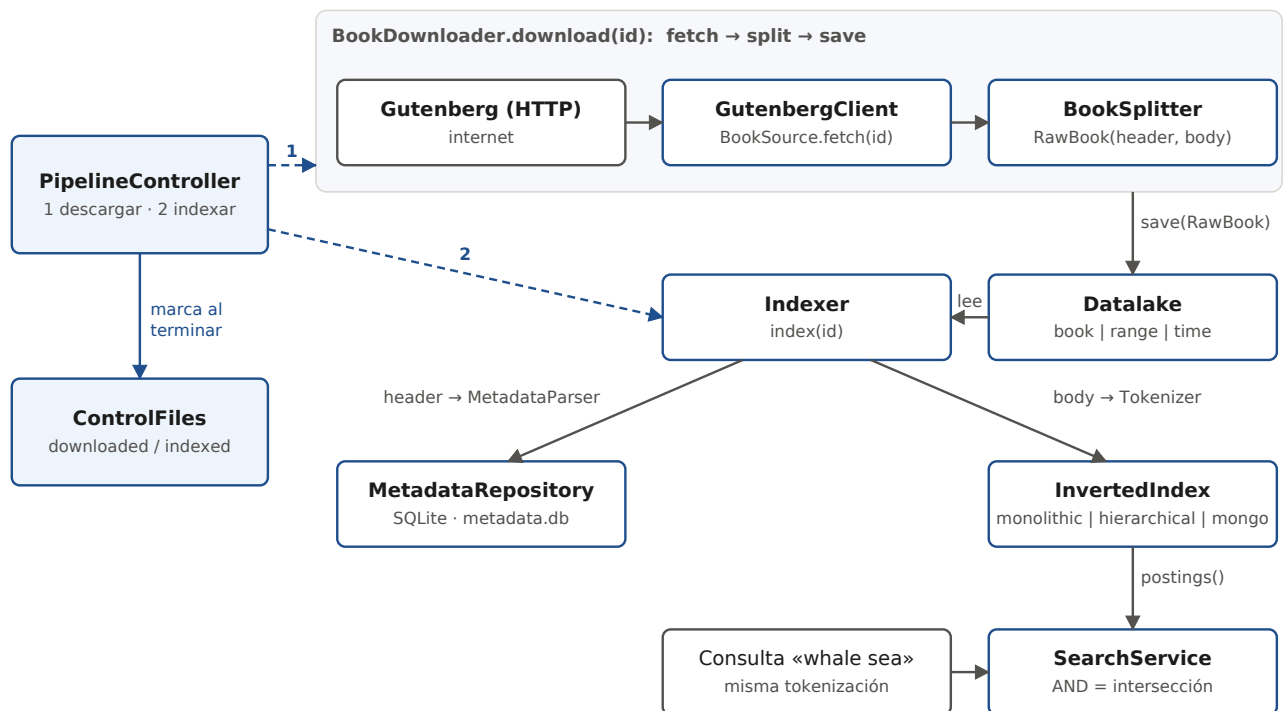
Lo que las tres implementaciones deben hacer igual. Conviene saberlo de memoria porque justifica muchas decisiones del código:

Sección	Regla
1. Dataset	<code>book_ids.txt</code> (ids a descargar, en orden), <code>queries.txt</code> (consultas del benchmark), <code>stopwords.txt</code> . Se ignoran líneas vacías y las que empiezan por #.
2. Separación	Marcadores <code>*** START OF THE/THIS PROJECT GUTENBERG EBOOK</code> y <code>*** END OF ...</code> . Si falta alguno, el libro se descarta. header = antes del START (trim); body = desde el final de la línea START hasta el END (trim). <code>\r\n</code> → <code>\n</code> .
3. Datalake	time: <code>YYYYMMDD/HH/<ID>.body.txt</code> · book: <code><ID>/body.txt</code> · range: <code><INI>-<FIN>/<ID>.body.txt</code> con <code>INI=(ID/1000)·1000</code> a 5 dígitos.
4. Metadatos	title, author, release_date, language por regex multilínea, primera coincidencia, sólo la primera línea, trim. Ausente → NULL. Tabla SQLite <code>books</code> con índices en author y title.
5. Tokenizador	Carácter a carácter: A-Z → minúscula; a-z y 0-9 forman token; cualquier otro carácter separa (incluidos acentos y apóstrofes). Tokens de longitud < 2 y stopwords fuera. Cada libro aporta el conjunto de términos (sin frecuencias).
6. Índice	monolithic: un JSON <code>{"term": [ids]}</code> · hierarchical: <code>inverted_index/<LETRA>/<term>.txt</code> · mongo: colección <code>inverted_index</code> con índice único en term. Posting lists ordenadas y sin duplicados.
7. Consultas	Se tokeniza la consulta con el mismo tokenizador; AND = intersección; ids ordenados.
8. Control	<code>control/downloaded_books.txt</code> e <code>indexed_books.txt</code> ; un id se añade sólo después de que la escritura terminó bien.

Sección	Regla
9. Benchmarks	CSV language, experiment, structure, dataset_size, repetition, metric, value, unit; 2 warmups + 5 medidas; sin red en las medidas.

2. Arquitectura

El diagrama muestra el recorrido de un libro. Las flechas indican «llama a» o «escribe en». Todas las cajas de almacenamiento son **interfaces** con varias implementaciones intercambiables.



SearchEngine crea y conecta todas las piezas con AppConfig + DatalakeFactory + InvertedIndexFactory (lo usan Main y la prueba end-to-end)

Figura 1. Componentes principales y flujo de datos.

2.1 Recorrido completo de un libro

- PipelineController.step()** mira los ficheros de control. No hay nada pendiente de indexar, así que elige el primer id de `book_ids.txt` que no esté descargado (p. ej. 1342).
- BookDownloader.download(1342)**: `GutenbergClient.fetch` descarga el .txt por HTTP → `BookSplitter.split` busca los marcadores y devuelve un `RawBook(id, header, body)` → `Datalake.save` escribe header y body con escritura atómica y devuelve un `BookLocation`.
- Sólo si todo fue bien, **ControlFiles.markDownloaded(1342)** añade «1342» a `downloaded_books.txt`.
- En el siguiente paso hay un libro descargado y no indexado. **Indexer.index(1342)** lee header y body del datalake, calcula TODO sin escribir nada (metadatos con `MetadataParser`, términos con `Tokenizer`), guarda los metadatos en SQLite y después añade el libro al índice y hace `flush()`.
- Sólo entonces **ControlFiles.markIndexed(1342)**.
- Una consulta «whale sea» pasa por **SearchService.search**: se tokeniza igual que los libros, se piden las posting lists de «whale» y «sea» al índice y se intersecan.

2.2 Capas y dependencias

Las dependencias van siempre hacia abajo: las capas altas conocen a las bajas, nunca al revés. Las piezas de negocio dependen de **interfaces** (*Datalake*, *InvertedIndex*, *MetadataRepository*, *BookSource*); sólo la capa de ensamblado sabe qué clase concreta se usa.

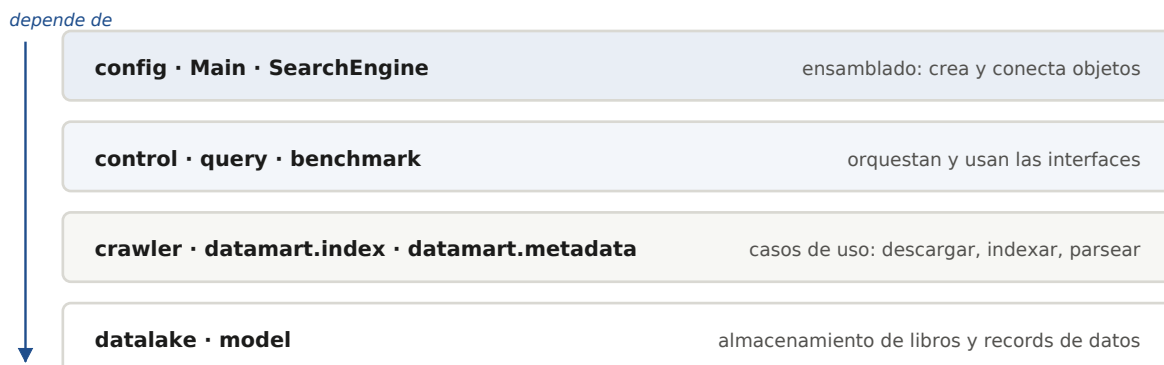


Figura 2. Capas: cada una sólo usa las de debajo.

2.3 Principios de diseño que se repiten

Principio	Dónde se ve	Por qué
Programar contra interfaces	<i>Datalake</i> , <i>InvertedIndex</i> , <i>MetadataRepository</i> , <i>BookSource</i>	Cambiar de estructura sin tocar a quien la usa; tests con dobles (lambdas, memoria).
Inyección por constructor	Todas las clases reciben sus piezas en el constructor.	Nada se crea «a escondidas»; en los tests se sustituye cualquier pieza.
Factory	<i>DatalakeFactory</i> , <i>InvertedIndexFactory</i>	Un nombre de configuración → una implementación, en un solo sitio.
Template method	<i>AbstractFileDatalake</i> : la subclase decide <i>dónde</i> , la base <i>cómo</i> se escribe y qué es un libro guardado.	La lógica delicada (atomicidad) existe una sola vez.
Escritura atómica	Datalake, JSON, ficheros de término, CSV: escribir <i>.tmp</i> y renombrar.	Un corte nunca deja un fichero a medias con el nombre bueno.
Idempotencia	<i>save</i> (upsert), <i>addDocument</i> (conjuntos), <i>mark*</i>	Repetir una operación tras un fallo es seguro: no duplica nada.
Marcar después de hacer	<i>PipelineController</i> : <i>mark*</i> sólo tras éxito.	El control nunca afirma algo que no ha pasado; como mucho se repite trabajo.
Records inmutables	<i>RawBook</i> , <i>BookLocation</i> , <i>BookMetadata</i> , <i>StepResult</i> , <i>AppConfig</i>	Datos sin comportamiento, <i>equals</i> automático, imposibles de modificar por accidente.

3. Organización del proyecto

3.1 Árbol de carpetas

Stage_1/	
├── README.md	descripción general del grupo
├── shared/	CONTRATO COMÚN (Java, Python y C)
│ ├── SPEC.md	reglas que no pueden variar entre lenguajes
│ ├── book_ids.txt	los 200 libros del dataset, en orden
│ ├── queries.txt	consultas AND del benchmark
│ └── stopwords.txt	47 palabras vacías en inglés
└── java/stage1/	← la implementación Java (se ejecuta desde aquí)
├── pom.xml	Maven: Java 17, JUnit 5, sqlite-jdbc, Jackson, driver MongoDB
├── config.properties	configuración por defecto (la lee AppConfig)
└── src/main/java/es/ulpgc/bigdata/	
├── Main.java	línea de comandos: pipeline search status config
├── SearchEngine.java	monta el sistema completo a partir de AppConfig
├── model/	RawBook, BookLocation, BookMetadata (records)
├── crawler/	BookSource, GutenbergClient, BookSplitter, BookDownloader
├── datalake/	Datalake + Book/Range/TimeBasedDatalake, AbstractFileDatalake
├── datamart/metadata/	MetadataParser, MetadataRepository, SQLite
├── datamart/index/	Tokenizer, InvertedIndex + 4 implementaciones, Indexer
├── query/	SearchService (AND)
├── control/	ControlFiles, PipelineController, StepResult, BookIdList
├── config/	AppConfig, DatalakeFactory, InvertedIndexFactory
└── benchmark/	BenchmarkRunner, CSV y los tres benchmarks
└── src/test/java/...	354 tests (JUnit 5), misma estructura de paquetes
└── data/	(generado, ignorado por git) datalake, datamarts, control
└── benchmarks/	(generado) results/{real,synthetic}/*.csv y work/ (ignorado)

3.2 Cifras

Qué	Cantidad
Clases e interfaces de producción	43 (≈ 4.700 líneas con comentarios)
Clases de test	33 (≈ 5.100 líneas) · 354 tests, todos en verde
Dependencias externas	sqlite-jdbc 3.53, jackson-databind 2.22, mongodb-driver-sync 5.13, JUnit 5.10
Historia	31 retos, un commit por reto en la rama java

3.3 Dónde vive cada parte del estado

Todas las rutas las construye `AppConfig` a partir de `data.dir` (por defecto `data/`, relativo a `java/stage1`). Ninguna otra clase escribe rutas.

Estado	Fichero / base	Quién lo escribe
Texto de los libros	<code>data/datalake/<estructura>/...</code>	<code>Datalake.save</code>
Metadatos	<code>data/datamarts/metadata.db</code> (tabla books)	<code>SQLiteMetadataRepository</code>
Índice monolithic	<code>data/datamarts/inverted_index.json</code>	<code>MonolithicJsonIndex.flush</code>
Índice hierarchical	<code>data/datamarts/inverted_index/<L>/<term>.txt</code>	<code>HierarchicalFolderIndex.flush</code>
Índice mongo	BD <code>search_engine</code> , colección <code>inverted_index</code> (proceso mongod)	<code>MongoInvertedIndex.flush</code>
Progreso	<code>data/control/downloaded_books.txt</code> , <code>indexed_books.txt</code>	<code>ControlFiles.mark*</code>

Estado	Fichero / base	Quién lo escribe
Resultados de benchmarks	<code>benchmarks/results/java_<experimento>.csv</code>	<code>CsvResults.write</code>

4. Cómo se ejecuta

4.1 Requisitos

- **Java 17 o superior** (el `pom.xml` compila para 17; se ha probado con OpenJDK 21).
- **Maven** (`mvn`). El README menciona un wrapper `mvnw` que no existe en el repositorio: hay que usar el Maven instalado.
- **MongoDB** sólo si se usa el índice `mongo` o su benchmark. Sin Mongo, sus tests se *saltan* (no fallan) y el benchmark lo omite con un aviso.
- **Internet** sólo para el comando `pipeline` (descarga de Gutenberg). Tests y benchmarks no usan la red.

Arrancar MongoDB con Docker (el contenedor del proyecto se llama `stage1-mongo`):

```
docker run -d --name stage1-mongo -p 127.0.0.1:27017:27017 mongo:8.2.12
docker start stage1-mongo          # las siguientes veces
```

El contenedor original se creó con **docker compose** desde `java/stage1`, pero el fichero `docker-compose.yml` no está en el repositorio. El `docker run` de arriba es equivalente.

4.2 Compilar y pasar los tests

```
cd Stage_1/java/stage1
mvn compile          # compila
mvn test             # 354 tests (los de Mongo se saltan si no está arrancado)
mvn test -Dtest=EndToEndTest # sólo la prueba de sistema
```

4.3 La aplicación: comandos de Main

Siempre desde `java/stage1` (las rutas por defecto son relativas a esa carpeta):

```
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="config"
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="pipeline 400"
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="search whale sea"
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="status"
```

Comando	Qué hace
<code>config</code>	Muestra la configuración efectiva (tras aplicar fichero, entorno y -D).
<code>pipeline [pasos]</code>	Da como máximo <i>pasos</i> pasos (10 por defecto). Un paso = una descarga o una indexación, así que cada libro necesita 2 pasos: <code>pipeline 400</code> procesa los 200 libros. Se puede interrumpir y relanzar: continúa donde se quedó.
<code>search <consulta></code>	Búsqueda AND con el índice activo; muestra id y título (de SQLite).
<code>status</code>	Tamaño del dataset, descargados, indexados y pendientes de indexar.

Ejemplo real de salida:

```
$ ... -Dexec.args="pipeline 4"
datalake=time index=monolithic
DOWNLOADED 1342
INDEXED 1342
DOWNLOADED 84
INDEXED 84

$ ... -Dexec.args="search whale sea"
1 libros para "whale sea" (index=monolithic)
84 Frankenstein; or, the modern prometheus
```

4.4 Configuración: cambiar de estructura sin tocar código

Cada valor sale de cuatro fuentes; la de abajo gana: (1) valores por defecto de `AppConfig`, (2) `config.properties` o el fichero de `--config`, (3) la variable de entorno `MONGO_URI`, (4) propiedades `-Dclave=valor`.

```
# config.properties
data.dir = data
shared.dir = ../../shared
benchmarks.dir = benchmarks
datalake.structure = time           # book | range | time
index.structure = monolithic       # monolithic | hierarchical | mongo | memory
mongo.uri = mongodb://localhost:27017
mongo.database = search_engine
mongo.collection = inverted_index
http.connect.timeout.seconds = 10
http.request.timeout.seconds = 15
```

```
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="pipeline 10" \
-Ddatalake.structure=book -Dindex.structure=hierarchical
```

Un valor mal escrito (`index.structure=hierarchcal`, una clave con errata o un timeout 0) hace que el programa falle **al arrancar** con un mensaje que enumera las opciones válidas.

Cuidado al cambiar de estructura con datos ya existentes. Los ficheros de control son comunes a todas las estructuras: si 84 se descargó con `time` y luego se cambia a `book`, el control dice que 84 ya está hecho y nunca llegará al nuevo datalake ni al nuevo índice. Solución práctica: borrar `data/` al cambiar. Solución de diseño (pendiente de acordar en grupo porque se aparta de la sección 8 del SPEC): un directorio de control por combinación de estructuras.

4.5 Los benchmarks

El capítulo 7 usa los 200 libros reales. Sin red: se descargan con el pipeline en estructura `book` y esa carpeta se pasa a los benchmarks:

```
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="pipeline 400" \
-Ddatalake.structure=book
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.benchmark.DatalakeBenchmark \
-Dexec.args="data/datalake/book"
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.benchmark.IndexBenchmark \
-Dexec.args="50,100,200 data/datalake/book"
mvn -q exec:java -Dexec.mainClass=es.ulpgc.bigdata.benchmark.MetadataBenchmark \
-Dexec.args="1000,10000,100000"
```

- `pipeline 400` = 200 descargas + 200 indexaciones. Sin la ruta, los benchmarks de datalake e índice usan libros sintéticos; el de metadatos lo es siempre. Sin Mongo, se omite con un aviso.
- Cada ejecución sobrescribe `benchmarks/results/java_*.csv`: los resultados se archivan a mano en `results/real/` y `results/synthetic/`. Para medir memoria con menos ruido, ejecutar con `java -cp` en vez de `mvn exec:java`.

5. Clases y responsabilidades

Para cada clase: qué es, su responsabilidad (lo único que hace), sus métodos importantes y el porqué de su diseño. El orden sigue el recorrido de los datos.

5.1 Paquete model: los datos que viajan

RawBook

record · model

Un libro recién descargado y separado, **antes** de guardarlo: `id`, `header`, `body`. Es lo que produce `BookSplitter` y consume `Datalake.save`.

Por qué así: Un record es inmutable y trae `equals/hashCode/toString` gratis: ideal para datos que sólo se pasan de una pieza a otra.

BookLocation

record · model

Dónde quedó guardado un libro: `id`, `headerPath`, `bodyPath`. No contiene texto, sólo rutas. Lo devuelven `save` y `locate`.

Por qué así: Separa «dónde está» de «qué contiene»: el `Indexer` lee el texto sólo cuando lo necesita.

BookMetadata

record · model

Metadatos de un libro: `id`, `title`, `author`, `language`, `releaseDate` y las dos rutas del datalake. Un campo ausente en el header es `null` (el SPEC dice NULL).

Por qué así: Guardar las rutas en SQLite enlaza la base de metadatos con el datalake: desde un resultado de búsqueda se llega al texto.

5.2 Paquete crawler: descargar y separar

BookSource

interfaz funcional · crawler

«De dónde sale el texto de un libro»: `Optional<String> fetch(int id)`. Vacío si no existe (404); un fallo de red es una excepción.

Por qué así: Permite probar todo sin red: en los tests es una lambda con libros inventados. La prueba end-to-end usa una que cuenta las llamadas.

GutenbergClient

clase · crawler · implementa BookSource

Descarga `https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt` con el `HttpClient` de Java. Sólo descarga: ni separa ni guarda.

- Constructor (`connectTimeout`, `requestTimeout`): los timeouts vienen de `AppConfig`.
- `fetch`: HTTP 2xx → texto; otro código o error de red → vacío.

Por qué así: Responsabilidad única: si Gutenberg cambia de URL sólo se toca esta clase.

BookSplitter

clase · crawler

Separa el texto crudo en header y body según el SPEC y descarta el footer. Normaliza `\r\n` a `\n`. Si falta un marcador devuelve vacío (el libro se descarta).

- El body empieza al final de la *línea* del marcador START (no justo tras el patrón), para no arrastrar el título que va en esa línea.
- Busca el END sólo después del START.

Por qué así: Es una función pura (sin I/O): se prueba con cadenas y es idéntica en los tres lenguajes.

BookDownloader

clase · crawler

Componente compuesto: `fetch` → `split` → `save`. Devuelve el `BookLocation` o vacío si el libro no está disponible o no tiene marcadores; en ese caso no escribe nada.

Por qué así: No toca los ficheros de control: eso lo decide quien lo llama (`PipelineController`). Así se puede reutilizar y probar aislado.

5.3 Paquete datalake: guardar los libros

Datalake

interfaz · datalake

Contrato de cualquier organización física: `name()`, `save(RawBook)`, `locate(id)`, `listBookIds()`. El resto del sistema sólo conoce esta interfaz.

- `locate` sólo devuelve libros **completos** (header y body con su nombre definitivo).
- `listBookIds` devuelve ids ordenados y sin repetir; ignora `.tmp` y basura.

Por qué así: Un test de contrato (`DatalakeContractTest`, 36 casos) se ejecuta contra las tres implementaciones: garantiza que se comportan igual.

AbstractFileDatalake

clase abstracta · datalake

Lo común a los tres datalakes de ficheros. Las subclases sólo deciden **dónde** va cada libro; aquí se define **cómo** se escribe y **qué** cuenta como libro guardado.

- `writeBook`: header primero y body después; un libro sólo existe cuando están los dos.
- `writeAtomically`: crear carpeta → escribir `x.tmp` → `move(ATOMIC_MOVE)` a `x`. Si el sistema no permite movimiento atómico, mueve normal.
- `isCompleteBook`, `parseCanonicalId` (rechaza `0084`, `-5`, ids de más de 9 dígitos).

Por qué así: Patrón *template method*. La parte delicada (atomicidad, definición de libro completo) está una sola vez y no puede divergir entre estructuras.

BookBasedDatalake

clase · datalake · estructura «book»

Una carpeta por libro: `<root>/<id>/header.txt` y `body.txt`.

Por qué así: La más simple: `locate` calcula la ruta directamente. Inconveniente: la raíz tiene tantas entradas como libros (200 en el benchmark; con millones sería un problema).

RangeBasedDatalake

clase · datalake · estructura «range»

Carpetas por rangos de 1000 ids: `01000-01999/1342.body.txt`.

Por qué así: La ruta también se calcula a partir del id (lookup directo) y ninguna carpeta pasa de 2000 ficheros. `listBookIds` comprueba además que cada fichero está en su rango.

TimeBasedDatalake

clase · datalake · estructura «time»

Carpetas por fecha y hora de la descarga: `20260930/19/84.body.txt`. Recibe un `Clock` para poder simular el tiempo en tests y benchmarks.

- `save` lee el reloj una sola vez (header y body en la misma carpeta aunque cambie la hora).
- `locate` no puede calcular la ruta: recorre las carpetas de la más reciente a la más antigua. Si un id está repetido, gana la copia más reciente.
- Formatos `yyyyMMdd` y `HH` (no `YYYY`, que es año ISO de semana, ni `hh`, que es 12 h).

Por qué así: Refleja cuándo llegó cada libro (útil para ingesta incremental), a costa de un lookup lento: se ve en el benchmark.

DatalakeStats

record · datalake

Foto de ocupación de una carpeta: directorios, ficheros, bytes y máximo de entradas en un directorio. La usan los benchmarks.

5.4 Paquete datamart.metadata: metadatos en SQLite

MetadataParser

clase · datamart.metadata

Extrae title, author, release_date y language del header con las regex del SPEC. No lee ficheros: recibe texto y el `BookLocation` (para el id y las rutas).

- Regex con `MULTILINE`: `^` y `$` marcan principio y fin de cada línea.
- release_date descarta lo que va entre corchetes: «March 3, 2001 [eBook #10]» → «March 3, 2001».
- Un valor vacío tras el trim se trata como ausente (`null`).

MetadataRepository

interfaz · datamart.metadata

Almacén de metadatos: `save`, `saveAll`, `findById`, `findByAuthor`, `findByTitle`, `count`, `clear`, `close`.

- `save/saveAll` son idempotentes (guardar otra vez un id lo actualiza).
- `saveAll` es todo o nada.
- Las búsquedas son por igualdad exacta y devuelven resultados ordenados por id.

Por qué así: `close()` se redeclara sin `throws Exception` para poder usarlo en `try-with-resources` sin capturar nada.

SqliteMetadataRepository

clase · *datamart.metadata*

Implementación con JDBC sobre un fichero SQLite. Mantiene **una** conexión abierta hasta `close()`. Crea la carpeta y el esquema al abrir.

- **Upsert:** `INSERT ... ON CONFLICT(book_id) DO UPDATE` → reindexar un libro no falla ni duplica.
- `saveAll`: una transacción con `addBatch/executeBatch` y un único commit; `rollback` si algo falla.
- Siempre `PreparedStatement` (nunca concatenar SQL: evita inyección y errores con comillas).

Por qué así: Envuelve `SQLException` en `MetadataRepositoryException`: quien usa el repositorio no depende de JDBC.

SqliteSchema

clase de utilidad · *datamart.metadata*

El `CREATE TABLE books` y los índices `idx_books_author` e `idx_books_title`, todo con `IF NOT EXISTS` (se puede ejecutar en cada apertura). También construye la URL JDBC.

MetadataRepositoryException

excepción no comprobada · *datamart.metadata*

Error del almacén de metadatos; envuelve la `SQLException` original.

5.5 Paquete datamart.index: el índice invertido

Un **índice invertido** guarda, para cada término, la lista de libros que lo contienen (su *posting list*).

Es «invertido» porque el libro contiene términos, y el índice da la vuelta: del término a los libros.

Ejemplo: `whale` → `[84, 2701]`, `sea` → `[84, 1342, 2701]`.

Tokenizer

clase · *datamart.index*

Convierte texto en términos según el SPEC: recorre carácter a carácter; `A-Z` → minúscula; `a-z` y `0-9` forman token; cualquier otro carácter separa. Descarta tokens de longitud `< 2` y stopwords.

- `tokenize`: todos los tokens con repeticiones.
- `uniqueTerms`: cada término una vez, en orden de primera aparición (lo que aporta cada libro).
- `fromStopwordsFile`: carga `shared/stopwords.txt`.

Por qué así: No usa `toLowerCase()` ni `Character.isLetter()`: siguen reglas Unicode que C no puede reproducir byte a byte. Por eso «café» da «caf» en los tres lenguajes.

InvertedIndex

interfaz · *datamart.index*

Contrato: `name`, `addDocument(id, terms)`, `postings(term)`, `flush`, `clear`, `diskUsageBytes`, `close`.

- `postings`: ordenada, sin repetidos, inmodificable; vacía si el término no existe.
- `postings` ve lo añadido aunque no se haya hecho `flush`.
- `addDocument` es idempotente; `clear` vacía también el disco.
- Tras `flush`, cerrar y reabrir conserva todo.

Por qué así: `InvertedIndexContractTest` (33 casos) verifica estas reglas en todas las implementaciones.

Implementación	Qué hace flush	Dónde vive el índice
<code>InMemoryInvertedIndex</code>	Nada (no persiste).	<code>HashMap<String, TreeSet<Integer>></code> en memoria.
<code>MonolithicJsonIndex</code>	Reescribe el JSON completo (tmp + mover).	Todo en memoria (<code>TreeMap</code>); se carga entero al abrir.
<code>HierarchicalFolderIndex</code>	Reescribe sólo los ficheros de los términos que cambiaron.	Un fichero por término; en memoria sólo lo pendiente.
<code>MongoInvertedIndex</code>	Un <code>bulkWrite</code> con un <code>upsert</code> + <code>\$addToSet</code> por término.	En el servidor MongoDB; en memoria sólo lo pendiente.

InMemoryInvertedIndex

clase · `datamart.index` · «memory»

Mapa término → `TreeSet` de ids. `computeIfAbsent(term).add(id)`: el `TreeSet` mantiene los ids ordenados y sin repetir. Sirve de referencia «perfecta» en tests y benchmarks.

MonolithicJsonIndex

clase · `datamart.index` · «monolithic»

Todo el índice en un `TreeMap<String, TreeSet<Integer>>` y en disco un único JSON (Jackson). Al abrir carga el fichero entero; `flush` lo reescribe entero si hay cambios (`dirty`).

- Si el JSON está corrupto, lanza excepción en lugar de empezar vacío: si no, el siguiente flush borraría el índice bueno.

Por qué así: Consultas rapidísimas (todo en memoria) y un fichero simple; pero actualizar cuesta lo mismo que escribir todo el índice, y el índice entero debe caber en la RAM.

HierarchicalFolderIndex

clase · `datamart.index` · «hierarchical»

Un fichero por término, un id por línea: `B/boat.txt`. La carpeta es la primera letra en mayúscula; los términos que empiezan por dígito van a la carpeta del dígito.

- `addDocument` acumula en `pending`; `flush` lee, une y reescribe (atómico) sólo los ficheros de los términos tocados.
- `postings` lee sólo el fichero del término pedido y le suma lo pendiente.
- Valida que el término sea `[a-z0-9]+` (evita rutas raras y problemas en sistemas que no distinguen mayúsculas).

Por qué así: Actualizar un libro sólo toca sus términos y la memoria es mínima; a cambio hay miles de ficheros pequeños (mucho espacio desperdiciado en bloques de 4 KB) y cada consulta es una lectura de disco.

MongoInvertedIndex

clase · `datamart.index` · «mongo»

Documentos `{"term": "boat", "postings": [10, 20]}` con índice único en `term`. Acumula en memoria y `flush` envía todo en un `bulkWrite` desordenado.

- `$addToSet` + `$each`: añade sólo ids que no estuvieran (idempotente).
- Mongo no garantiza el orden del array: `postings` ordena al leer.
- `diskUsageBytes` = `storageSize` + `totalIndexSize`, tras un `fsync` (sin él, `WiredTiger` puede no haber escrito aún los datos).
- Timeout de 3 s si Mongo no responde.

Por qué así: Actualizaciones baratas y un servidor que gestiona caché, compresión y concurrencia; el coste es un viaje por red en cada consulta.

Indexer

clase · datamart.index

Puente datalake → datamarts para un libro: `index(id)`.

- 1) Leer y calcular todo sin escribir nada (header, body, metadatos, términos).
- 2) Guardar metadatos. 3) `addDocument` + `flush`.
- Devuelve vacío si el libro no está en el datalake (y no toca nada).

Por qué así: Calcular antes de escribir evita estados a medias. Metadatos antes que índice: un libro con metadatos pero sin índice es invisible; lo contrario daría resultados sin título.

5.6 Paquete query: la búsqueda

SearchService

clase · query

Búsqueda AND: consulta → tokens (mismo tokenizador) → posting lists → intersección. Nunca lee libros.

- Si un término no tiene postings, devuelve [] sin pedir los demás (en Mongo o disco cada petición cuesta).
- Ordena las listas de la más corta a la más larga: el resultado nunca es mayor que la más corta.
- `intersect`: dos punteros sobre listas ordenadas, como mucho $n+m$ pasos ($O(n+m)$).

Por qué así: Funciona igual con cualquier `InvertedIndex`.

5.7 Paquete control: progreso y recuperación

BookIdList

clase de utilidad · control

Lee `shared/book_ids.txt`: ids en orden, sin repetidos; ignora vacías y #. Una línea que no es un id es un **error** (el fichero se escribe a mano y una errata no debe pasar desapercibida).

ControlFiles

clase · control

Los dos ficheros de control como **conjuntos**: `readyToIndex = downloaded - indexed`.

- Se cargan una vez; cada marca se añade al final (append), primero al disco y después a la memoria.
- Al cargar, sólo cuentan las líneas completas (terminadas en `\n`); un trozo a medias se recorta para que el siguiente append no forme un id falso (`«13» + «84\n» = «1384»`).

PipelineController

clase · control

Decide qué hacer y se lo pide a quien sabe: no descarga, no parsea, no indexa.

- `step()`: como mucho una operación. Prioridad: indexar lo descargado; si no, descargar el siguiente del dataset; si no, IDLE.
- Marca en el control **sólo después** de que la operación termine bien.

- Los libros que fallan se saltan en esta ejecución (`skippedThisRun`, sólo en memoria) y se reintentan al reiniciar.
- `runUntilIdle(max)`: repite pasos hasta IDLE o hasta el límite.

Por qué así: Todo lo necesario para continuar está en los ficheros de control; el proceso puede morir en cualquier momento sin perder ni duplicar nada.

StepResult

record · control

Qué hizo un paso: `action` (INDEXED, DOWNLOADED, NOT_AVAILABLE, MISSING_FROM_DATA LAKE, DOWNLOAD_FAILED, INDEX_FAILED, IDLE), `bookId` y `detail` (mensaje de error).

5.8 Paquete config y ensamblado

AppConfig

record · config

Toda la configuración y el **único** sitio que construye rutas (`datalakeDir()`, `metadataDb()`, `monolithicIndexFile()`, `controlDir()`...).

- `load()`: valores por defecto < fichero < `MONGO_URI` < -D.
- Valida en el constructor: estructura inexistente, clave desconocida o `timeout ≤ 0` → error al arrancar.
- `withDataDir`: misma configuración con otra carpeta (los benchmarks trabajan aparte).

Por qué así: Un test recorre `src/main/java` y falla si aparece una ruta de datos fuera de AppConfig.

DatalakeFactory

clase de utilidad · config

Traduce "book" | "range" | "time" a la clase correspondiente. Único `new XxxDatalake` de la aplicación. Nombre desconocido → excepción con las opciones válidas.

InvertedIndexFactory

clase de utilidad · config

Traduce "monolithic" | "hierarchical" | "mongo" | "memory" a un `InvertedIndex`, con las rutas y datos de conexión que da AppConfig.

SearchEngine

clase · raíz

El sistema completo montado a partir de un AppConfig: datalake, metadatos, índice, control, pipeline y búsqueda. Es el **único** sitio donde se conectan las piezas.

- `open(config)`: con Gutenberg real (lo usa Main).
- `open(config, source)`: con cualquier `BookSource` (la prueba end-to-end, sin red).
- `close()`: cierra índice y SQLite (el segundo aunque falle el primero).

Por qué así: Si el test montara las piezas por su cuenta, probaría su montaje y no el del programa.

Main

clase · raíz

Capa fina de línea de comandos: interpreta `[--config fichero] comando`, carga `AppConfig`, pide el sistema a `SearchEngine` e imprime. Sin lógica de negocio. Errores de configuración → mensaje y código de salida 2.

5.9 Paquete benchmark

BenchmarkRunner

clase · benchmark

Repite una tarea y la mide: para cada repetición ejecuta `setup()` (**no** medido) y luego `task()` entre dos `System.nanoTime()`. Descarta los warmups y devuelve una fila por medida.

Por qué así: `nanoTime` es un reloj monótono pensado para medir intervalos; `currentTimeMillis` es la hora de pared y puede saltar (NTP).

BenchmarkRow

record · benchmark

Una fila del CSV común. Valida que los textos no lleven comas ni comillas (así el CSV no necesita escapado y lo leen igual Python y C) y formatea números con punto decimal y como máximo 3 decimales.

CsvResults

clase de utilidad · benchmark

Escribe `benchmarks/results/java_<experimento>.csv` entero de una vez, con `tmp` + mover: si el benchmark falla, el CSV anterior queda intacto.

BenchmarkBooks

clase de utilidad · benchmark

De dónde salen los libros de los benchmarks (nunca de la red): de un datalake existente, `synthetic` (16 palabras, para el datalake) o `syntheticZipf` (30.000 palabras con distribución de Zipf, para el índice; las palabras de las consultas en rangos 10, 40, 90... para que tengan selectividades distintas).

SimulatedClock

clase · benchmark

Reloj que avanza 6 minutos cada vez que se consulta: simula una ingesta de 10 libros por hora, de forma reproducible, para que la estructura `time` reparta los libros en carpetas como en la realidad.

DatalakeBenchmark · MetadataBenchmark · IndexBenchmark

clases · benchmark

Los tres benchmarks; se explican en detalle en el capítulo siguiente.

6. Estrategia de pruebas

354 tests con JUnit 5. Siguen tres niveles:

Nivel	Qué comprueba	Ejemplos
Unitario	Una clase aislada, con dobles para sus dependencias.	<code>TokenizerTest</code> (17), <code>MetadataParserTest</code> (14), <code>BookSplitterTest</code> , <code>ControlFilesTest</code> (13)
Contrato	Las mismas pruebas contra todas las implementaciones de una interfaz.	<code>DatalakeContractTest</code> (36), <code>InvertedIndexContractTest</code> (33)
Integración	Varias piezas reales juntas, con disco y bases de datos de verdad.	<code>IndexerTest</code> , <code>PipelineControllerTest</code> , <code>SqliteMetadataRepositoryTest</code> , <code>MongoInvertedIndexTest</code>
Sistema (end-to-end)	El programa entero, montado igual que en Main, sin red.	<code>EndToEndTest</code> (6)

6.1 La prueba end-to-end (reto 31)

- 1 Tres libros inventados con formato Gutenberg real (`\r\n`, marcadores, pie de licencia), ids 10, 1500 y 2003 en tres rangos distintos.
- 2 Una `BookSource` falsa que cuenta las llamadas: la prueba no depende de internet.
- 3 `SearchEngine.open(config, fake)` → pipeline hasta IDLE: 3 descargas + 3 indexaciones.
- 4 Metadatos: título, autor, idioma, fecha sin «[eBook #10]», y que la ruta del body en SQLite es la del datalake.
- 5 10 consultas AND con resultado conocido: varios términos, mayúsculas y puntuación, números, sólo stopwords, y palabras que sólo están en título, autor o pie (no deben aparecer: sólo se indexa el body).
- 6 Se cierra todo, se comprueba en disco dónde está cada parte del estado, se reabre con una fuente que **falla si se usa** y se repiten las consultas: el resultado debe ser idéntico y nada se vuelve a descargar.

Se repite con `time+monolithic`, `book+hierarchical`, `range+monolithic` y `book+mongo`. Un segundo test simula que el proceso muere sin llamar a `close()`: el estado sigue ahí porque se hace persistente con cada `flush` y cada marca, no al cerrar.

Para comprobar que la prueba sirve se introdujeron dos fallos a propósito en el montaje: olvidar las stopwords (la consulta «the of» devolvió [2003]) y guardar el control en otra carpeta en cada arranque («tras el reinicio se volvió a descargar el libro 10»). La prueba detectó ambos.

7. Benchmarks

7.1 Metodología común

- **Warmups y medidas:** 2 repeticiones descartadas (calientan el JIT de Java, las cachés del disco y las conexiones) y 5 medidas. En las tablas se da la **mediana** de las 5, que resiste bien los valores atípicos (una pausa del recolector de basura, por ejemplo).
- **Setup fuera de la medida:** vaciar, preparar datos, abrir conexiones... va en `setup()`, que el runner no cronometra. Hay un test que lo verifica con un setup lento a propósito.
- **Mismos datos para todos:** cada estructura recibe exactamente la misma lista y en el mismo orden.
- **Sin red:** la descarga nunca está dentro de la medida. Los libros reales se leen de un datalake `book` que el pipeline ya llenó; los sintéticos se generan con semilla fija (reproducibles).
- **Tamaños crecientes:** cada N usa los N primeros libros del mismo dataset (prefijos), así las curvas son comparables. Con libros reales el orden es el de los ids (`BenchmarkBooks.fromDatalake` los lee ordenados), no el de `book_ids.txt`.
- **Verificación:** tras medir se comprueba el resultado (libros encontrados, filas insertadas, posting lists iguales a una referencia). Un benchmark que mide algo incorrecto falla en lugar de escribir números.
- **Salida:** un CSV por experimento con el formato común del SPEC, comparable con Python y C. Cada ejecución escribe `benchmarks/results/java_<experimento>.csv`; los resultados se archivan en `results/real/` o `results/synthetic/` según los datos usados.

Entorno de estas medidas: Linux, 12 hilos, 15 GB de RAM, SSD NVMe, OpenJDK 21, MongoDB 8.2 en Docker (local). Los valores absolutos dependen de la máquina; lo que importa son las **proporciones** y cómo cambian al crecer N.

7.2 Datos: libros reales y sintéticos

Los benchmarks se han ejecutado con dos juegos de datos. Los resultados principales de esta memoria son los **reales**: son los que pide el SPEC (los mismos libros en Java, Python y C) y los únicos con un vocabulario de verdad. Los sintéticos, de la primera versión de la memoria, se conservan como referencia y se comparan con los reales en el apartado 7.6.

Benchmark	Datos reales (<code>results/real/</code>)	Datos sintéticos (<code>results/synthetic/</code>)
Datalake	Los 200 libros de <code>shared/book_ids.txt</code> descargados de Gutenberg (129,0 MB).	200 libros de 300 KB con 16 palabras distintas (61,5 MB).
Índice	Prefijos de 50, 100 y 200 de esos libros. Con 200: 129.356 términos y 1.581.064 postings.	Libros de 5.000 palabras elegidas con Zipf entre 30.000. Con 200: 29.716 términos.
Metadatos	— (200 filas son pocas para ver escalar SQLite; ver 7.4).	Filas generadas: 1.000, 10.000 y 100.000.

Los 200 ids son los 15 originales más los libros en inglés más descargados de Gutenberg y ficción clásica. Antes de añadirlos se comprobó que todos se descargan y tienen los marcadores START/END, para que el pipeline no descarte ninguno. Muchos son clásicos con ids bajos: **110 de los 200 están por debajo de 1.000**, algo que se nota en la estructura range.

Para reproducirlos: descargar con `pipeline 400` y `-Ddatalake.structure=book`, y pasar `data/datalake/book` a `DatalakeBenchmark` e `IndexBenchmark` (apartado 4.5).

7.3 Benchmark del datalake (reto 27)

Datos: los 200 libros reales (129,0 MB, los mismos bytes en las tres estructuras). La estructura time usa el `SimulatedClock` (10 libros por hora).

Experimento	Qué se mide exactamente	Métricas
<code>datalake_write</code>	Guardar los N libros en un datalake vacío (setup: borrar la carpeta).	elapsed (ms), throughput (libros/s)
<code>datalake_lookup</code>	locate() de todos los ids en un orden aleatorio fijo (semilla 42).	elapsed, per_lookup (µs)
<code>datalake_incremental</code>	Con el 90 % conocido y el 10 % nuevo: listBookIds() menos los conocidos.	elapsed, detected
<code>datalake_recovery</code>	A 1 de cada 10 libros se le deja el body como <code>.tmp</code> (corte simulado); se mide detectar los que faltan y volver a guardarlos.	elapsed, recovered, lost, duplicates
<code>datalake_storage</code>	Tras guardar todo: ficheros, carpetas, máx. entradas por carpeta, bytes y bytes reservados en bloques.	files, directories, max_entries_per_dir, bytes, allocated_bytes

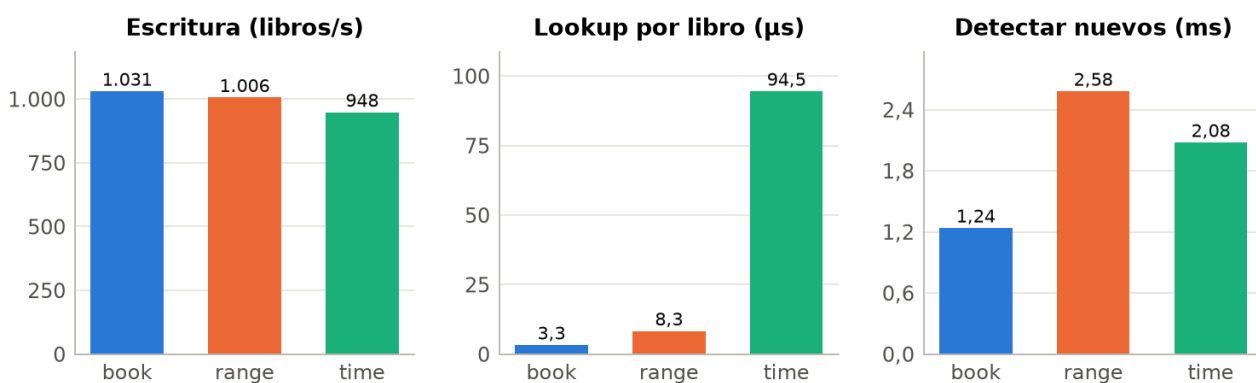


Figura 3. Datalake con los 200 libros reales (mediana de 5 medidas). Barras más altas = más rápido sólo en la primera gráfica; en las otras dos, más bajo es mejor.

Métrica	book	range	time
Escritura total (ms)	194,0	198,8	210,9
Escritura (libros/s)	1.031	1.006	948
Lookup por libro (µs)	3,3	8,3	94,5
Detectar nuevos (ms)	1,24	2,58	2,08
Recuperación (ms)	23,8	25,2	19,8
Libros recuperados	20	20	20
Perdidos / duplicados	0 / 0	0 / 0	0 / 0
Carpetas	200	47	21
Máx. entradas en una carpeta	200	220	20
Bytes (MB)	129,0	129,0	129,0
Bytes reservados (MB)	130,9	130,3	130,2

Interpretación

- **Lookup:** book (3,3 μ s) y range (8,3 μ s) **calculan** la ruta a partir del id: una comprobación de fichero. time (94,5 μ s, unas 29 veces más que book) no puede calcularla y recorre las carpetas de hora de la más reciente a la más antigua. Su coste crece con el número de horas de ingesta.
- **Escritura:** las tres tardan casi lo mismo, con un 9 % de diferencia entre la más rápida y la más lenta (194–211 ms, $\approx$ 1.000 libros/s): el coste lo dominan escribir 129 MB y los renombrados atómicos, no la organización de carpetas. Con los datos sintéticos el orden entre las tres era otro (time era la más rápida), señal de que estas diferencias son ruido.
- **Incremental y recuperación:** todas detectan exactamente los 20 libros nuevos (10 %) y recuperan los 20 dañados sin pérdidas ni duplicados, gracias a la definición única de «libro completo» (un `.tmp` nunca cuenta). Recuperar cuesta ahora 20–25 ms ($\approx$ 8 ms con sintéticos) porque se reescriben libros reales, que miden de media más del doble ($\approx$ 645 KB frente a 300 KB).
- **Almacenamiento, lo que cambia con datos reales:** los bytes son idénticos en las tres, pero 110 de los 200 ids están entre 0 y 999, así que range mete 110 libros (220 ficheros) en la carpeta `00000-00999`: su carpeta más llena tiene **más** entradas que la raíz de book (200). Con ids sintéticos repartidos uniformemente parecía la más equilibrada (máximo 67). range garantiza como mucho 2.000 ficheros por carpeta, pero no reparte bien si los ids se concentran. time limita cada carpeta a lo que llega en una hora (20 ficheros), sea cual sea el id.
- **Conclusión:** book/range si se busca por id (lo habitual al indexar y al mostrar resultados); time si importa procesar por orden de llegada. range sigue combinando lookup directo con un límite de ficheros por carpeta, pero lo bien que reparte depende de los ids.

7.4 Benchmark de metadatos (reto 28)

Datos: metadatos sintéticos: el libro i tiene autor «Author $i/10$ » y título «Title $i/2$ ». Así cada autor tiene siempre 10 libros y cada título 2, sea cual sea N : al crecer N sólo crece la tabla, no el número de resultados. El parseo del header **no** está en el tiempo. Tamaños: 1.000, 10.000, 100.000 filas.

Se comparan dos variantes: `sqlite` (con los índices del SPEC sobre author y title) y `sqlite_no_index` (sin ellos; `book_id` sigue siendo PRIMARY KEY), para ver qué aportan los índices.

Es el único benchmark que sigue con datos sintéticos, y es intencionado: no lee libros sino filas ya construidas, así que el texto real no influye, y la diferencia entre índice y *full scan* sólo se aprecia con miles de filas, no con 200. Sus resultados están en `results/synthetic/` y no dependen del dataset.

Experimento	Qué se mide	Métricas
<code>metadata_insert</code>	saveAll en una base vacía, en lotes de 1000 (cada lote, una transacción).	elapsed (ms), throughput (filas/s)
<code>metadata_query</code>	1000 consultas de cada tipo, con valores que existen (semilla fija). Se mide cada tipo por separado.	find_by_id, find_by_author, find_by_title (ms total) y *_avg (μ s/consulta)

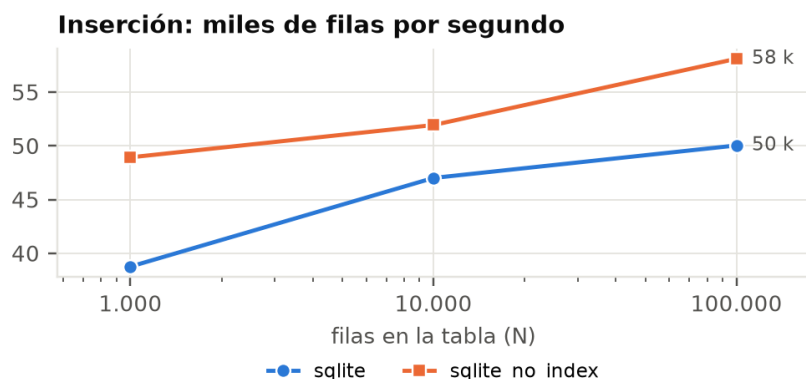


Figura 4. Inserción: miles de filas por segundo frente a N (eje X logarítmico).

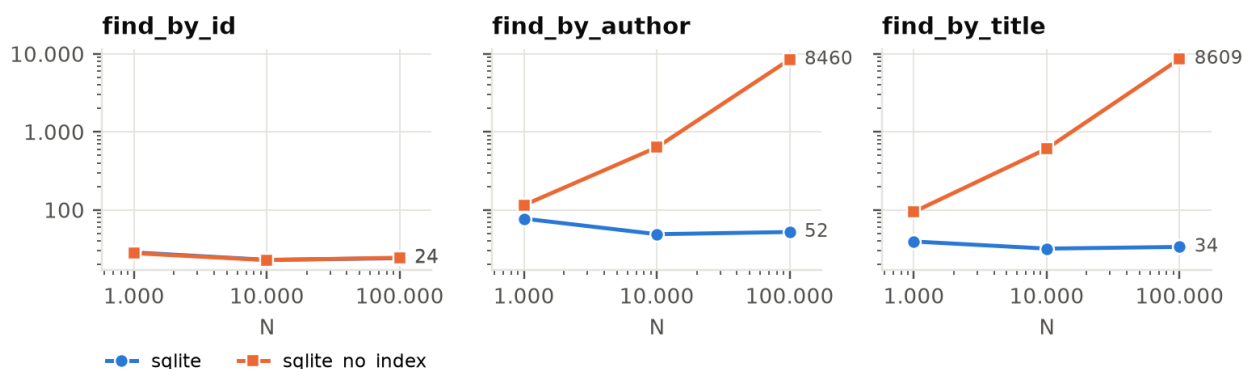


Figura 5. Coste medio por consulta en μ s frente a N (ambos ejes logarítmicos). En `find_by_id` las dos líneas coinciden (ambas variantes usan la PRIMARY KEY), por eso sólo se ve la naranja.

Métrica (N = 100.000)	sqlite	sqlite_no_index	Relación
Inserción (filas/s)	50.036	58.103	× 1,2
find_by_id (μ s)	24,0	24,4	× 1,0
find_by_author (μ s)	52,0	8.459,6	× 162,6
find_by_title (μ s)	33,5	8.608,9	× 256,8

Interpretación

- **Consultas por author/title:** sin índice SQLite hace un *full scan* (lee toda la tabla), así que el coste crece de forma lineal con N: de 115,1 μ s con 1.000 filas a 8.459,6 μ s con 100.000. Con índice (un árbol B) es logarítmico y casi plano: 77,2 μ s $\rightarrow$ 52,0 μ s. Con 100.000 filas el índice es $\approx$ 163 veces más rápido.
- **find_by_id:** igual en las dos variantes, porque ambas tienen la PRIMARY KEY (en SQLite, la tabla está organizada por `book_id`). Sus $\approx$ 24 μ s son prácticamente el coste fijo de una consulta JDBC (preparar la sentencia, ejecutarla y convertir la fila): es el suelo de cualquier consulta en esta implementación.
- **Inserción:** mantener los dos índices tiene un precio: cada fila nueva actualiza también dos árboles B. Por eso la variante sin índices inserta más rápido. Es el trade-off clásico: índices = lecturas rápidas a cambio de escrituras más lentas y más disco.
- **Lotes:** insertar en transacciones de 1000 filas es clave; con autocommit, cada fila sería una transacción con su escritura a disco.

7.5 Benchmark de los índices invertidos (reto 29)

Datos: los 200 libros reales, con el tokenizador del SPEC. Tamaños: 50, 100 y 200 libros (los de id más bajo primero). Con 200 libros el índice lógico tiene 129.356 términos y 1.581.064 postings (pares término-libro), **idénticos en los tres backends**. Cada libro aporta de media 7.905 términos distintos.

Reglas de equidad: todos los libros se tokenizan una vez antes de medir (el tokenizador no contamina); antes de cada repetición `freshIndex` hace `clear` y comprueba que el backend quedó vacío; después de medir `verify` compara el índice con uno en memoria construido con los mismos términos. Mongo usa su propia base `search_engine_bench`.

Experimento	Qué se mide exactamente	Métricas
<code>index_build</code>	De índice vacío a índice persistido: N <code>addDocument</code> con términos ya calculados + un <code>flush</code> . No incluye leer libros, tokenizar, vaciar ni conectar.	elapsed, throughput (libros/s)
<code>index_query</code>	El índice se construye y se reabre ; se mide 100 veces la carga de <code>queries.txt</code> con <code>SearchService</code> (incluye tokenizar la consulta).	elapsed, per_query (μs)
<code>index_update</code>	Índice con N-k libros (k = 10 %), reabierto; se mide añadir los k libros con un <code>flush</code> por libro, como hace el Indexer.	elapsed, per_book (ms)
<code>index_memory</code>	Heap de Java (tras pedir GC) con el índice construido y recién abierto.	heap_after_build, heap_after_open (bytes)
<code>index_disk</code>	Tras construir: bytes lógicos, ficheros y bloques reservados.	bytes, files, allocated_bytes, terms, postings

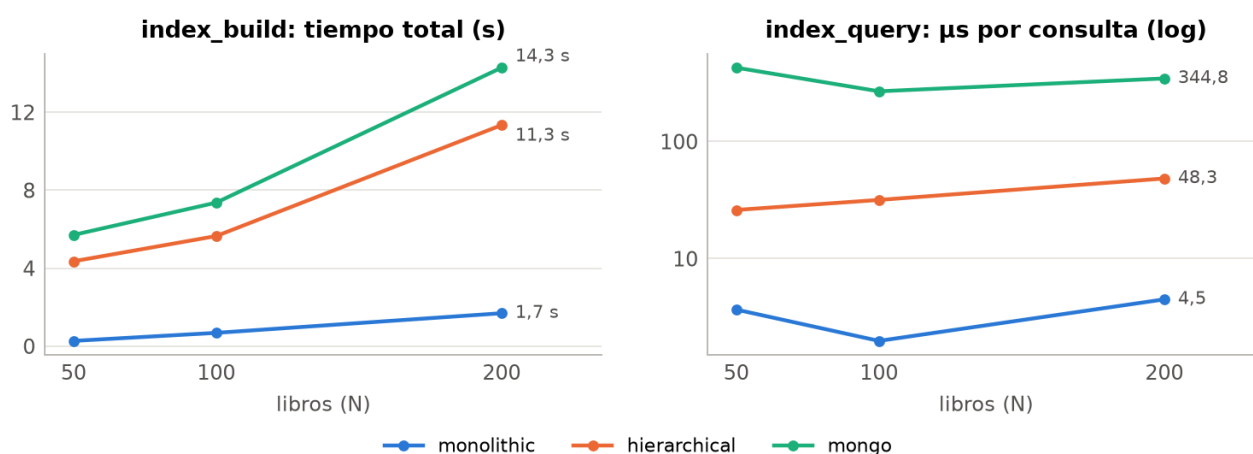


Figura 6. Construcción (tiempo total) y consulta (μs por consulta, eje logarítmico) frente al número de libros.

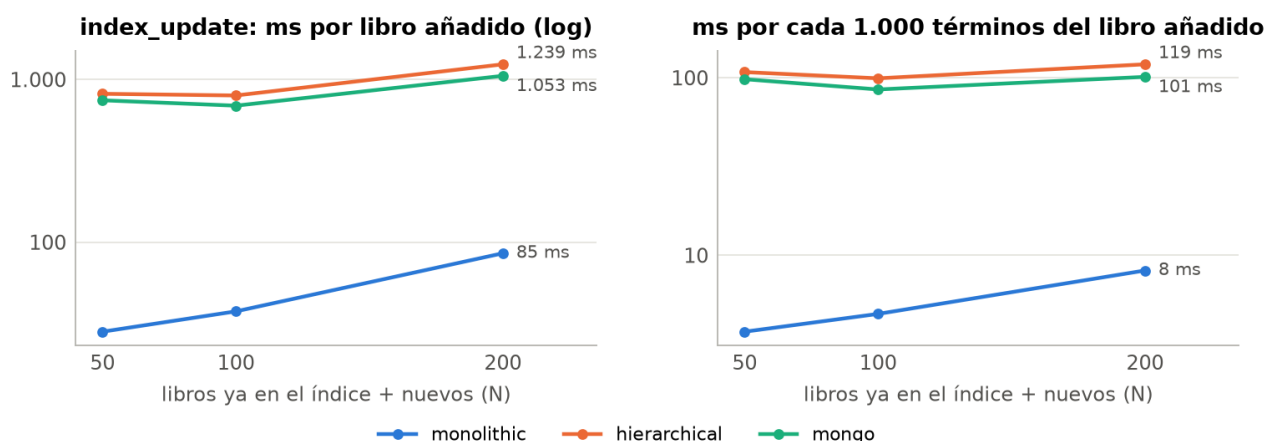


Figura 7. Actualización. Izquierda: coste por libro añadido; sólo monolithic crece con el tamaño del índice. Derecha: el mismo coste dividido por los términos del libro añadido; hierarchical y mongo son casi constantes ($\approx 0,1$ ms por término).

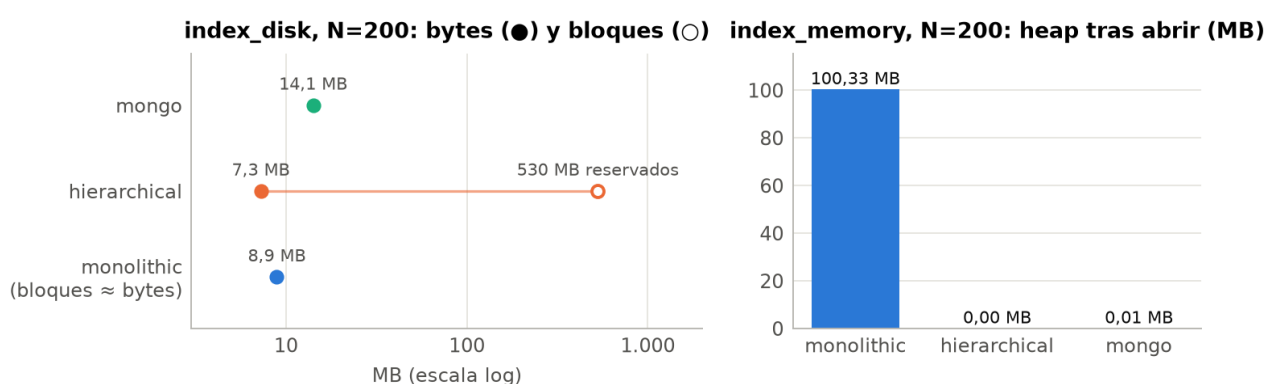


Figura 8. Con 200 libros: espacio en disco (izquierda; ● bytes de datos, ○ bloques reservados por el sistema de ficheros) y heap de Java con el índice recién abierto (derecha).

Métrica (mediana)	N=50	N=100	N=200
Build (s) · monolithic	0,27	0,68	1,69
Build (s) · hierarchical	4,35	5,65	11,34
Build (s) · mongo	5,70	7,36	14,29
Consulta (μ s) · monolithic	3,7	2,0	4,5
Consulta (μ s) · hierarchical	26,0	31,7	48,3
Consulta (μ s) · mongo	426,4	267,9	344,8
Update por libro (ms) · monolithic	28,1	37,5	85,4
Update por libro (ms) · hierarchical	816,3	797,2	1.239,0
Update por libro (ms) · mongo	744,5	690,0	1.052,7
Términos medios del libro añadido	7.592	8.037	10.415
Disco (MB) · monolithic	1,90	3,75	8,89
Disco (MB) · hierarchical	1,15	2,74	7,25
Disco (MB) · mongo	3,98	7,27	14,13
Bloques (MB) · monolithic	1,9	3,8	8,9
Bloques (MB) · hierarchical	241,1	323,0	530,0
Heap al abrir (MB) · monolithic	18,50	48,29	100,33

Métrica (mediana)	N=50	N=100	N=200
Heap al abrir (MB) · hierarchical	0,00	0,00	0,00
Heap al abrir (MB) · mongo	0,01	0,02	0,01
Términos	58.834	78.820	129.356
Postings	360.970	759.087	1.581.064

Interpretación

- **Construcción:** monolithic es con diferencia la más rápida (1,69 s con 200 libros): todo ocurre en memoria y al final se escribe un solo fichero. hierarchical (11,3 s) escribe un fichero por término (129.356 ficheros, cada uno con su tmp + renombrado: $\approx 88 \mu\text{s}$ por fichero) y mongo (14,3 s) envía un upsert por término mientras el servidor mantiene su índice único. El throughput de monolithic baja de 187 a 119 libros/s al crecer N: el TreeMap y el JSON final son cada vez más grandes.
- **Consultas:** monolithic responde en $4,5 \mu\text{s}$ porque la posting list ya está en memoria; hierarchical lee un fichero por término y crece con N porque cada fichero tiene más ids ($26,0 \rightarrow 48,3 \mu\text{s}$); mongo ($268\text{--}426 \mu\text{s}$) está dominado por el viaje de ida y vuelta al servidor por término, y por eso no sigue una tendencia clara con N.
- **Actualizaciones, el resultado clave:** el **flush** de monolithic reescribe el JSON **entero**, así que su coste por libro crece con el índice: $28,1 \rightarrow 37,5 \rightarrow 85,4 \text{ ms}$, unos 10 ms por cada MB de JSON. hierarchical y mongo sólo tocan los términos del libro añadido, y su coste es casi constante **por término** ($\approx 0,1 \text{ ms}$ en los dos; figura 7, derecha). Su subida en N=200 ($0,8 \rightarrow 1,2 \text{ s}$ y $0,7 \rightarrow 1,1 \text{ s}$ por libro) no se debe al índice, sino a que esos 20 libros son más largos (10.415 términos de media frente a $\approx 7.592\text{--}8.037$).
- **¿Cuándo deja de compensar monolithic?** Con estos tamaños aún gana por un factor 12, pero es la única curva que crece con el índice. A $\approx 10 \text{ ms}$ por MB, alcanzaría el $\approx 1 \text{ s}$ por libro de mongo cuando el JSON ronde los 100 MB; con $\approx 44 \text{ KB}$ de JSON por libro, eso es en torno a **2.000–2.500 libros** (una extrapolación, no una medida). Con datos sintéticos la estimación era ≈ 2.800 .
- **Disco:** los bytes de datos son del mismo orden (7,3–14,1 MB), pero mongo es ahora el que más ocupa (14,1 MB frente a 8,9 MB de monolithic): cuenta sus datos más su índice único sobre term, que con 4,4 veces más términos que en los datos sintéticos pesa mucho más. hierarchical ocupa **530 MB reales para 7,3 MB de datos** ($\times 73$): cada uno de sus 129.356 ficheros ocupa como mínimo un bloque de 4 KB ($129.356 \times 4.096 \text{ B} = 530 \text{ MB}$, justo lo medido).
- **Memoria:** monolithic mantiene el índice entero en el heap: 100 MB con 200 libros para un JSON de 8,9 MB ($\approx 63 \text{ bytes}$ por posting: cada id es un Integer dentro de un TreeSet). Crece de forma lineal ($18,5 \rightarrow 48,3 \rightarrow 100,3 \text{ MB}$, $\approx 0,5 \text{ MB}$ por libro), así que con el catálogo completo de Gutenberg (más de 70.000 libros) necesitaría decenas de GB. hierarchical y mongo quedan casi a cero tras el flush: los datos están en disco o en otro proceso.

Limitaciones de la medida de memoria

- `System.gc()` es una petición, no una orden; el valor es una estimación con ruido de unos KB y puede salir negativo.
- hierarchical y mongo dejan 0,5–2,1 MB en el heap tras construir aunque su flush vacía lo pendiente: `HashMap.clear()` borra las entradas pero conserva la tabla interna ya dimensionada, cuyo tamaño es una potencia de 2 que depende del número de términos: $131.072 \text{ posiciones} \times 4 \text{ bytes} = 524.288 \text{ bytes}$ con 50 y 100 libros, y $524.288 \text{ posiciones} \times 4 \text{ bytes} = 2.097.152 \text{ bytes}$ con 200, justo lo medido. Al reabrir el índice, ese objeto es nuevo y el heap queda en unos cientos de bytes.

- Para Mongo sólo se ve el heap del **cliente** (driver, conexiones, lo pendiente). Los datos viven en el proceso **mongod**, con su propia caché WiredTiger, que es compartida por todas las colecciones y crece hasta un límite configurado, no según lo que necesite el índice. Por eso la RAM de un proceso externo no es comparable con el heap de Java.

7.6 Datos reales frente a sintéticos: qué cambia

Los mismos experimentos con N=200, con los datos sintéticos de la primera versión ([results/synthetic/](#)) y con los 200 libros reales ([results/real/](#)). La figura 9 muestra cuántas veces mayor es cada métrica con libros reales.

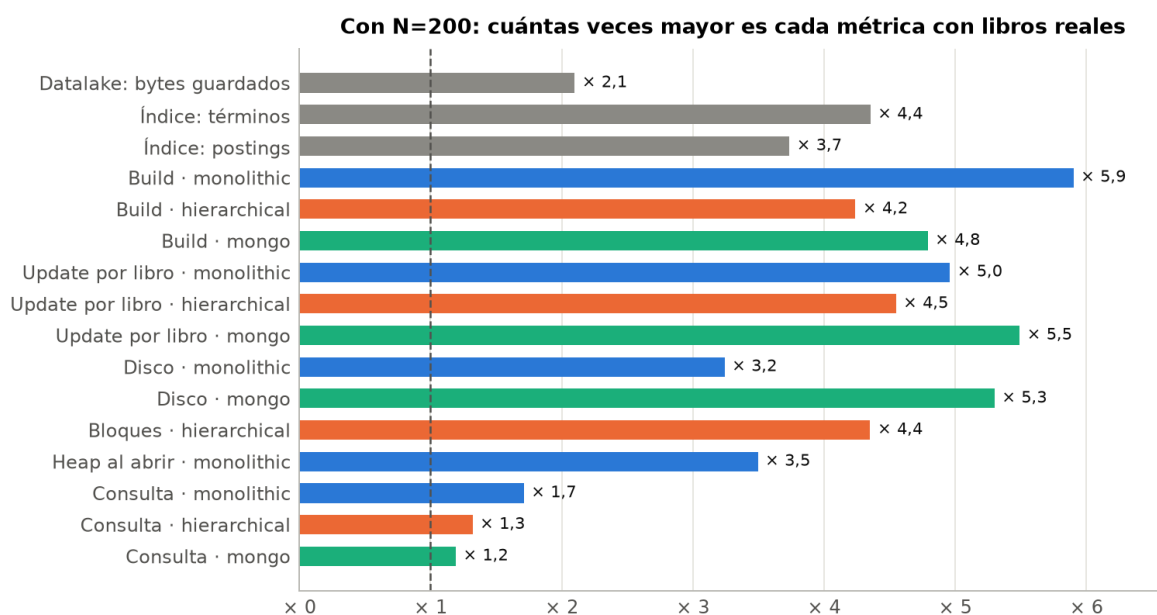


Figura 9. Factor real / sintético con N=200 (la línea discontinua es × 1: sin cambio). En gris, el tamaño de los datos; en color, cada backend.

Métrica (N = 200)	Sintético	Real	Real / sintético
Datalake: bytes guardados	61,5 MB	129,0 MB	× 2,1
Datalake: lookup · time	143,3 µs	94,5 µs	× 0,7
Índice: términos	29.716	129.356	× 4,4
Índice: postings	423.261	1.581.064	× 3,7
Build · monolithic	0,29 s	1,69 s	× 5,9
Build · hierarchical	2,68 s	11,34 s	× 4,2
Build · mongo	2,98 s	14,29 s	× 4,8
Update por libro · monolithic	17,2 ms	85,4 ms	× 5,0
Update por libro · hierarchical	272,4 ms	1.239,0 ms	× 4,5
Update por libro · mongo	191,7 ms	1.052,7 ms	× 5,5
Disco · monolithic	2,74 MB	8,89 MB	× 3,2
Disco · mongo	2,67 MB	14,13 MB	× 5,3
Bloques · hierarchical	121,8 MB	530,0 MB	× 4,4
Heap al abrir · monolithic	28,71 MB	100,33 MB	× 3,5
Consulta · monolithic	2,6 µs	4,5 µs	× 1,7

Métrica (N = 200)	Sintético	Real	Real / sintético
Consulta · hierarchical	36,6 μ s	48,3 μ s	× 1,3
Consulta · mongo	289,3 μ s	344,8 μ s	× 1,2

Interpretación

- **Las conclusiones se mantienen.** En todos los experimentos de índice el orden entre backends es el mismo: monolithic construye, consulta y actualiza más rápido; hierarchical desperdicia más disco; monolithic es el único que llena el heap. En el datalake, time sigue siendo la única estructura con lookup lento.
- **Las magnitudes, no.** Construir, actualizar, el disco y la memoria cuestan entre 3 y 6 veces más con libros reales. El generador sintético limita el vocabulario a 30.000 palabras y cada libro a 5.000 palabras ($\approx$ 2.116 términos distintos), mientras que un libro real aporta $\approx$ 7.905 (nombres propios, números, palabras raras) y el vocabulario no deja de crecer: 129.356 términos con 200 libros.
- **Las consultas apenas cambian (× 1,2-1,7):** una consulta sólo toca las posting lists de sus pocos términos, así que depende mucho menos del tamaño total del índice que construir o actualizar.
- **Lo que sólo se ve con datos reales:** el reparto desigual de los ids (range con 220 entradas en una carpeta), que mongo ocupe más disco que monolithic y que el coste de un update dependa del libro concreto que se añade.
- **Lección:** los datos sintéticos sirven para **comparar** estructuras entre sí; para **dimensionar** (cuánto disco, cuánta RAM, cuánto tarda) hacen falta datos reales.

7.7 Resumen: qué estructura elegir

Si lo importante es...	Datalake	Índice
Buscar por id / mostrar resultados	book o range	—
Procesar por orden de llegada	time	—
Carpetas pequeñas aunque los ids se concentren	time	—
Consultas lo más rápidas posible, índice pequeño	—	monolithic
Añadir libros continuamente (ingesta incremental)	range	hierarchical o mongo
Índice más grande que la RAM (más de $\approx$ 2.000 libros reales)	—	hierarchical o mongo
Poco espacio en disco	cualquiera	monolithic (hierarchical desperdicia bloques; mongo ocupa más con vocabulario real)
Varios procesos consultando / escalar	—	mongo

8. Preguntas de defensa

Preguntas que pueden hacer, con una respuesta corta y correcta. Muchas vienen de los propios retos.

8.1 Diseño general

¿Por qué tantas interfaces?

Para que quien usa algo no dependa de cómo está hecho. `Indexer` recibe un `InvertedIndex` y funciona igual con JSON, carpetas o Mongo. Además permite tests con dobles (una lambda como `BookSource`, un índice en memoria) y tests de contrato contra todas las implementaciones.

¿Qué problema resuelve una factory aquí?

Separar *usar* un objeto de *crearlo*. Sólo la factory sabe que «hierarchical» es `HierarchicalFolderIndex` en `datamarts/inverted_index/`. Sin ella ese switch estaría copiado en Main y en cada benchmark, y añadir una estructura obligaría a tocarlos todos.

¿Qué lógica no debería acabar en Main?

Ninguna decisión del dominio: qué libro toca (`PipelineController`), cómo separar (`BookSplitter`), tokenizar o intersecar (`Tokenizer`, `SearchService`), dónde vive cada fichero (`AppConfig`) ni qué clase corresponde a cada nombre (factories). Main sólo lee argumentos, conecta e imprime.

¿Cómo cambias de time a book o de monolithic a hierarchical?

Con `-Ddatalake.structure=book -Dindex.structure=hierarchical` o editando `config.properties`. No se toca código. (Con datos existentes, borrar `data/` por los ficheros de control compartidos.)

¿Por qué records?

Son datos inmutables con `equals/hashCode/toString` automáticos. `RawBook`, `BookLocation` o `BookMetadata` sólo transportan datos; que no se puedan modificar evita errores cuando pasan de una pieza a otra.

8.2 Robustez y recuperación

¿Por qué escribir en .tmp y luego renombrar?

Un renombrado dentro del mismo sistema de ficheros es atómico: el fichero con su nombre definitivo está completo o no existe. Si el proceso muere mientras escribe, sólo queda un `.tmp`, que nunca cuenta como dato válido.

¿Por qué el header se guarda antes que el body?

Un libro sólo «existe» cuando están los dos. Si el corte ocurre entre ambos, falta el body y el libro no cuenta como guardado: se volverá a descargar.

¿Por qué se marca el control después de la operación y no antes?

Si se marcara antes y el proceso muriera, el control diría «hecho» sobre algo que no está: se perdería el libro para siempre. Marcando después, lo peor que pasa es repetir la operación, y eso es seguro porque todo es idempotente.

¿Qué significa idempotente y dónde se usa?

Que repetir la operación no cambia el resultado. `save` de metadatos es un `upsert`; `addDocument` usa conjuntos; Mongo usa `$addToSet`; las marcas no se duplican; guardar un libro dos veces sobrescribe.

¿Qué pasa si se va la luz a mitad de indexar?

El libro no se marcó como indexado. Al reiniciar, `readyToIndex` lo vuelve a dar y se reindexa: el `upsert` sobrescribe los metadatos y `addDocument` no duplica ids.

¿Qué estado debe sobrevivir a un reinicio?

Lo que no se puede reconstruir sin repetir trabajo: datalake, metadatos, índice y ficheros de control. Lo que vive sólo en memoria (pendiente de flush, libros fallidos en esta ejecución) puede perderse porque el control garantiza que se repite sin duplicar.

8.3 Índice y búsqueda

¿Qué es un índice invertido?

Un mapa término → lista ordenada de libros que lo contienen. Permite responder «¿qué libros contienen X?» sin leer los libros.

¿Cómo se hace la búsqueda AND y cuánto cuesta?

Se tokeniza la consulta, se piden las posting lists, se ordenan de más corta a más larga y se intersecan con dos punteros (listas ordenadas): $O(n+m)$ por intersección. Si una lista está vacía se devuelve [] sin pedir más.

¿Por qué el tokenizador no usa `toLowerCase()`?

Porque sigue reglas Unicode que C no puede reproducir byte a byte, y el SPEC exige el mismo resultado en los tres lenguajes. Sólo se tratan A-Z, a-z y 0-9; todo lo demás separa.

¿Por qué monolithic se comporta distinto en updates?

Su flush reescribe el JSON entero aunque sólo cambie un libro: el coste de cada actualización crece con el tamaño del índice (28 → 85 ms por libro de 50 a 200 libros reales, ≈ 10 ms por MB de JSON). hierarchical y mongo sólo escriben los términos del libro nuevo.

¿Por qué hierarchical ocupa tanto disco?

Porque tiene un fichero por término (129.356 con los 200 libros reales) y el sistema de ficheros reserva como mínimo un bloque de 4 KB por fichero, aunque contenga 3 bytes: 530 MB reservados para 7,3 MB de datos.

8.4 Benchmarks

¿Qué mide exactamente `index_build`?

El tiempo desde un índice vacío y abierto hasta tener los N libros persistidos: N `addDocument` con los términos ya calculados + un `flush`. No incluye leer libros, tokenizar, vaciar el índice ni abrir la conexión.

¿Por qué se repitieron los benchmarks con libros reales?

Porque el SPEC pide que los tres lenguajes midan con los mismos libros, y porque los sintéticos subestiman: su vocabulario está limitado a 30.000 palabras. Con libros reales el índice tiene 4,4 veces más términos y construir, actualizar, el disco y la memoria cuestan de 3 a 6 veces más. El orden entre estructuras no cambió: los sintéticos valen para comparar, no para dimensionar.

¿Por qué el benchmark de metadatos sigue siendo sintético?

Mide SQLite con filas ya construidas (el parseo del header no está en el tiempo), así que el texto de los libros no influye. Y para ver la diferencia entre índice y *full scan* hacen falta miles de filas, no 200.

Si el update de hierarchical y mongo no depende del índice, ¿por qué sube en N=200?

Porque depende de los términos del libro que se añade: los 20 libros añadidos en N=200 tienen 10.415 términos de media frente a ≈ 8.000 en los otros tamaños. Dividido por término, el coste es casi constante ($\approx 0,1$ ms).

¿Por qué la RAM de un proceso externo no es comparable con el heap de Java?

Con Mongo los datos están en otro proceso; el heap de Java sólo ve el driver. Además la caché de WiredTiger es compartida, crece hasta un límite configurado y guarda datos comprimidos en páginas, mientras que en Java un `TreeSet<Integer>` gasta decenas de bytes por id (≈ 63 medidos en monolithic).

¿Para qué sirven los warmups?

La primera ejecución incluye compilar el código con el JIT, cargar clases, llenar cachés de disco y abrir conexiones. Descartarlas mide el régimen estable.

¿Por qué nanoTime y no currentTimeMillis?

nanoTime es monótono y está pensado para medir intervalos; currentTimeMillis es la hora del sistema y puede saltar si se ajusta el reloj.

¿Por qué la mediana y no la media?

Una sola repetición anómala (una pausa del GC) desplaza mucho la media y apenas afecta a la mediana.

¿Por qué en el datalake time el lookup es lento?

Porque la ruta depende de cuándo se descargó el libro, no de su id: hay que buscarlo recorriendo las carpetas de hora. book y range calculan la ruta directamente.

¿Qué aportan los índices de SQLite y qué cuestan?

Convierten la búsqueda por author/title de un recorrido completo (lineal en N) en una búsqueda en un árbol B (logarítmica). Cuestan inserciones más lentas y más espacio en disco.

8.5 Pruebas

¿Qué bugs no detectan los tests unitarios y sí una prueba de integración?

Los que están entre piezas: montaje (olvidar las stopwords al conectar el tokenizador), rutas (el control en otra carpeta), contratos entre piezas (indexar el header en vez del body), formato real (`\r\n`) y persistencia (algo que funciona en memoria pero no se guarda o no se relee).

¿Cómo compruebas que la prueba end-to-end no depende de la red?

Usa una BookSource falsa que cuenta las llamadas; tras el reinicio usa otra que lanza un error si se llama. GutenbergClient nunca se crea.

9. Limitaciones y trabajo futuro

- **Control compartido entre estructuras:** cambiar de estructura con datos existentes deja libros fuera del nuevo datalake/índice. Propuesta: un directorio de control por combinación (requiere acordar un cambio en la sección 8 del SPEC).
- **README desactualizado:** menciona `./mvnw` y `java -jar target/stage1.jar`, pero no hay wrapper ni jar ejecutable con dependencias. Se ejecuta con `mvn exec:java`.
- **docker-compose.yml** no está en la rama java (el contenedor se creó con compose).
- **Metadatos sólo con datos sintéticos:** datalake e índice ya se miden con los 200 libros reales; el benchmark de metadatos mide filas generadas, a propósito (ver 7.4).
- **Tamaños pequeños** (≤ 200 libros reales): suficientes para ver las tendencias y estimar cuándo deja de compensar monolithic ($\approx 2.000\text{--}2.500$ libros), pero ese punto es una extrapolación; convendría medir con miles de libros.
- **Prefijos por id:** cada N toma los ids más bajos, no una muestra aleatoria. Los ids bajos son clásicos antiguos y los últimos en entrar son más recientes y más largos (por eso sube el update en $N=200$). Una muestra aleatoria con semilla fija suavizaría las curvas.
- **Una máquina y una ejecución por configuración:** la mediana de 5 medidas resiste pausas puntuales, pero no la variación entre ejecuciones (se nota en las consultas de mongo).
- **Sin ranking:** la búsqueda devuelve ids ordenados, sin relevancia (no se guardan frecuencias, por decisión del SPEC). Es trabajo de las siguientes etapas.